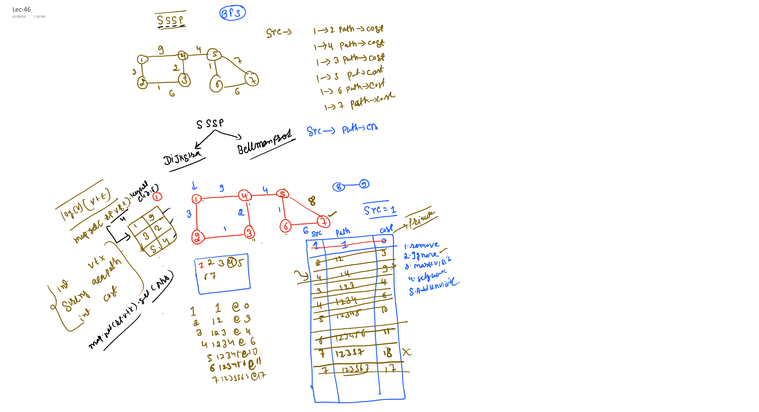
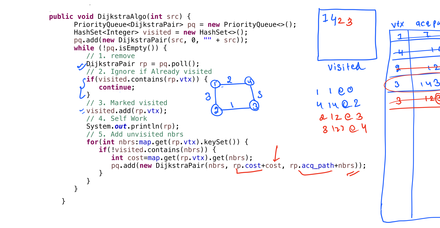
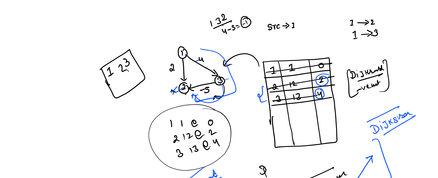
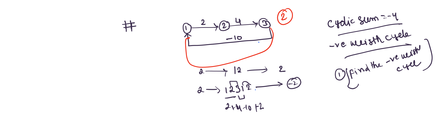
Lec-46

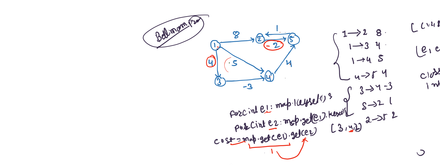


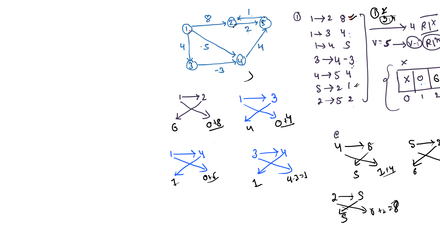
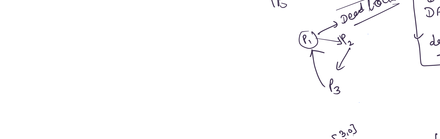

```
public void DijkstraAlgo(int src) {
      PriorityQueue<DijkstraPair> pq = new PriorityQueue<>();
      HashSet<Integer> visited = new HashSet<>();
      pq.add(new DijkstraPair(src, 0, "" + src));
      while (!pq.isEmpty()) {
        // 1. remove
        DijkstraPair rp = pq.poll();
        // 2. Ignore if Already visited
        if(visited.contains(rp.vtx)) {
          continue;
        }
        // 3. Marked visited
        visited.add(rp.vtx);
        // 4. Self Work
        System.out.println(rp);
        // 5. Add unvisited nbrs
        for(int nbrs:map.get(rp.vtx).keySet()) {
          if(!visited.contains(nbrs)) {
            int cost=map.get(rp.vtx).get(nbrs);
            pq.add(new DijkstraPair(nbrs, rp.cost+cost, rp.acq_path+nbrs));
          }
        }
      }
    }
```











```
public void BellManford(int src) {
      int v = map.size();
      int[] dis = new int[v + 1];
      for (int i = 1; i < dis.length; i++) {
        if (src != i) {
          dis[i] = 999999;
        }
      }

      List<EdgePair> ll = getAllEdge();
      for (int i = 1; i <= v; i++) {
        for (EdgePair e : ll) {
          if (dis[e.e2] > dis[e.e1] + e.cost) {
            dis[e.e2] = dis[e.e1] + e.cost;
          }
        }
      }

      for (int i = 1; i < dis.length; i++) {
        System.out.println(i + " " + dis[i]);
      }
    }
```